

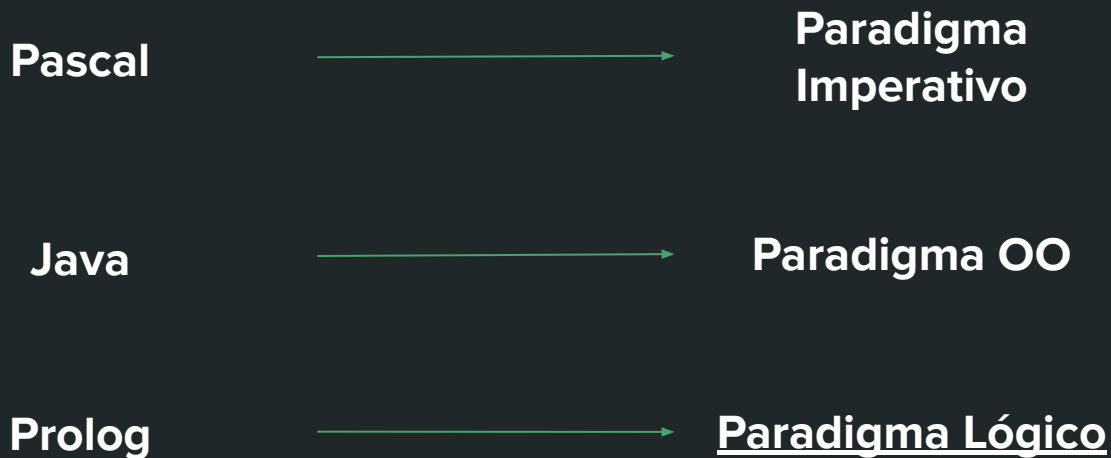
Introducción a PROLOG

Lenguajes Formales y Autómatas - UNS.
Por Diego Orbe.

Parte I: Programación Lógica

Paradigma de Programación Lógica

Un **paradigma de programación** es un enfoque o estilo para escribir y estructurar código:



Paradigma de Programación Lógica

El **paradigma de programación lógico** nos permite pensar los programas en términos de **premisas, reglas y consultas**:

Persona

- padre: Persona
- madre: Persona
- hijos: Persona[]

+ getPadre(): Persona

Juan.getPadre()

Paradigma OO

*Quiero armar un
árbol genealógico y
luego consultar
quien es el padre de
Juan*



?- padre(X,juan).

*“¿Es verdad que hay una
persona X que es padre
de Juan? Quien es?”*

Paradigma Lógico



Qué es Prolog?

Prolog es un lenguaje de programación donde usaremos conceptos de la Lógica (premisas, conclusiones, reglas) para resolver problemas computacionales.

Como vamos a ver, en Prolog es muy fácil representar y recuperar **conocimiento**, con el cual podemos razonar.



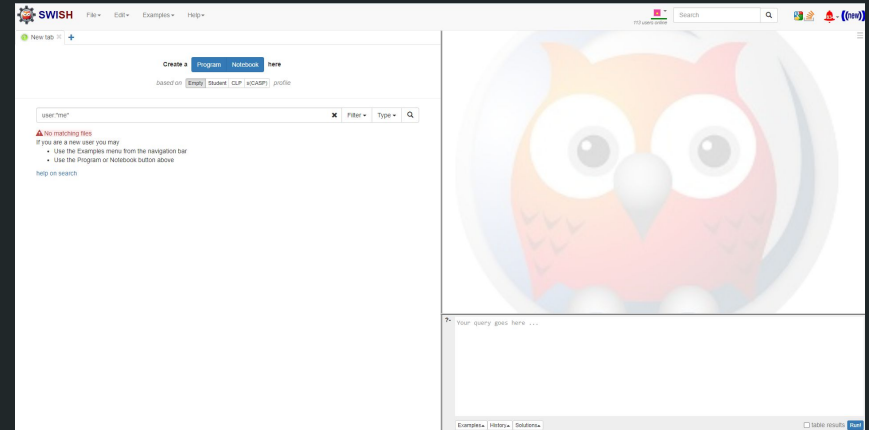
Inteligencia Artificial

Antes de empezar!



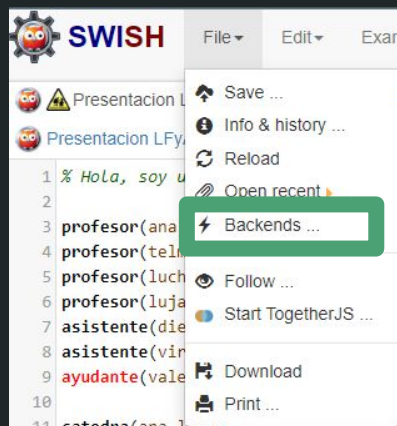
- En la materia utilizaremos el intérprete SWI-Prolog.
- Recomendamos usar su versión online en

<https://swish.swi-prolog.org/>



Antes de empezar!

Para evitar errores, recomendamos cambiar el backend de SWISH



SWISH backends

Backend	URL	Users	Alive
VU	https://swish.swi-prolog.org/	111	✓
dev1	https://swish2.swi-prolog.org/	2	✓
dev2	https://swish2.swi-prolog.org/	2	✓

Select backend

Elegir uno de estos dos.

Sintaxis básica de PROLOG.

Un programa PROLOG está compuesto por reglas y hechos, los cuales están compuestos por **términos**, los cuales pueden ser:

- **Variables:** se representan con identificadores que empiezan con **Mayúscula**.
Ejemplos: Ciudad, Mes, Nombre1.
Si no nos interesa usar el valor de la variable, podemos declararla como **variable anónima** empleando el **guión bajo** (**_**).
Ejemplo: _nombre , _ab11
- Las **constantes** representan valores fijos. Hay dos tipos de constantes: **números**, y **átomos**. Los **átomos** se representan con identificadores que empiezan con **minúscula**.
Ejemplos: bahia_blanca, octubre
- **Predicados:** se denota como un **átomo** (es decir, tiene un nombre que empieza con minúscula) y contiene una **secuencia de argumentos** dentro de paréntesis (puede aplicarse recursivamente).
*Ejemplos: madre(X,juan) , ciudad(bahia_blanca) ,
empleado(persona(juan,31), empresa(golitech, tecnologia)) .*

Términos en PROLOG - Semántica

En PROLOG, los **predicados** nos permiten relacionar argumentos (que pueden ser variables, átomos u otras estructuras):

padre(juan,tito) → “juan es **padre** de tito”

docente(X,lfy) → “X (que es una variable) es **docente** de lenguajes formales y autómatas.”

Ojo! el orden importa: **padre(juan,tito) ≠ padre(tito,juan)**

“telma es docente de una materia X”

“La empresa jpsolutions contrató a diego, que tiene 28 años”

“diego es docente en lfy”

Programa PROLOG

Un programa PROLOG está conformado por **reglas y hechos**.

- Los hechos son **términos**.
- Las reglas permiten relacionar **términos** entre sí y tienen la forma:

<conclusion> :- <antecedente>

¿Cómo se interpretan estos hechos/reglas?



Hechos

Reglas

```
% Hola, soy un programa Prolog!
```

```
profesor(ana) .  
profesor(telma) .  
profesor(lucho) .  
profesor(lujan) .  
asistente(diego) .  
asistente(vir) .  
ayudante(romi) .
```

```
catedra(ana,lfya) .  
catedra(telma,lfya) .  
catedra(diego,lfya) .  
catedra(vir,lfya) .
```

```
docente(X) :-  
    profesor(X) .
```

```
docente(X) :-  
    asistente(X) .
```

```
compañeros(X,Y) :-  
    catedra(X,Z) ,  
    catedra(Y,Z) .
```

Entendiendo los hechos.

Los **hechos** son términos que consideramos **verdaderos** incondicionalmente.

Consideremos el hecho **asistente(diego)**.

- **diego** es un término que utilizamos para representar a alguien.
- **asistente(X)** indica que X es asistente de alguna materia.

Luego, el hecho **asistente(diego)** indica que **diego** es asistente de alguna materia.

Hechos

```
% Hola, soy un programa Prolog!
```

```
profesor(ana) .  
profesor(telma) .  
profesor(lucho) .  
profesor(lujan) .  
asistente(diego) .  
asistente(vir) .  
ayudante(romi) .
```

```
catedra(ana,lfya) .  
catedra(telma,lfya) .  
catedra(diego,lfya) .  
catedra(vir,lfya) .
```

```
docente(X) :-  
    profesor(X) .
```

```
docente(X) :-  
    asistente(X) .
```

```
compañeros(X,Y) :-  
    catedra(X,Z) ,  
    catedra(Y,Z) .
```

Entendiendo las reglas.

Analicemos la regla

```
docente(X) :- profesor(X) .  
      <Conclusion>      :-      <Antecedente>
```

En prolog, las reglas se interpretan como
“Si se cumple el **antecedente** (1 o más términos),
entonces se cumple la **conclusión** (1 término)”

El predicado **docente(X)** lo utilizamos
indicar que **X es docente**.

El predicado **profesor(X)** lo utilizamos
para indicar que **X es profesor**.

Y por lo tanto...

Reglas

```
% Hola, soy un programa Prolog!
```

```
profesor(ana) .  
profesor(telma) .  
profesor(lucho) .  
profesor(lujan) .  
asistente(diego) .  
asistente(vir) .  
ayudante(romi) .
```

```
catedra(ana,lfy) .  
catedra(telma,lfy) .  
catedra(diego,lfy) .  
catedra(vir,lfy) .
```

```
docente(X) :-  
      profesor(X) .
```

```
docente(X) :-  
      asistente(X) .
```

```
compañeros(X,Y) :-  
      catedra(X,Z) ,  
      catedra(Y,Z) .
```

Entendiendo las reglas.

```
docente(X) :- profesor(X) .  
    <Conclusión>      :-      <Antecedente>
```

La regla se puede interpretar como:

Si X es profesor, entonces X es docente.

Si pensamos en términos de fórmulas bien formadas:

$\text{profesor}(X) \Rightarrow \text{docente}(X)$

Pensemos otros ejemplos!

Reglas

```
% Hola, soy un programa Prolog!
```

```
profesor(ana) .  
profesor(telma) .  
profesor(lucho) .  
profesor(lujan) .  
asistente(diego) .  
asistente(vir) .  
ayudante(romi)
```

```
catedra(ana,lfya) .  
catedra(telma,lfya) .  
catedra(diego,lfya) .  
catedra(vir,lfya) .
```

```
docente(X) :-  
    profesor(X) .
```

```
docente(X) :-  
    asistente(X) .
```

```
compañeros(X,Y) :-  
    catedra(X,Z) ,  
    catedra(Y,Z) .
```

Entendiendo las reglas.

“Si X tiene sueño **y** si hay yerba, entonces X tomara mate”



$\text{tiene_sueño}(X) \wedge \text{hay_yerba} \Rightarrow$
 $\text{toma_mate}(X)$



```
toma_mate(X) :-  
    tiene_sueño(X) ,  
    hay_yerba .
```

“Si X es un pájaro **o** un avión, entonces X vuela”

$(\text{pajaro}(X) \vee \text{avion}(X) \Rightarrow \text{vuela}(X))$



“Si X es un pájaro, entonces vuela”



“Si X es un avión, entonces vuela”



```
vuela(X) :-  
    pajaro(X) .  
vuela(X) :-  
    avion(X) .
```

Hagamos una actividad

Story time!

5:45 PM - Terminó de dar clases, estoy ____ de hambre, por suerte me guarde un Guaymallen en la Oficina.

Cuando abrí la heladera de la oficina, descubrí que mi Guaymallen no estaba...

Esto no puede quedar así...

Tengo que deducir quién pudo haberse robado mi alfajor:

- El culpable tiene que ser **sospechoso** y tener un **motivo** para robarlo:
 - a. El **culpable** **tenía hambre** y por eso lo robó, o
 - b. El **culpable** **no me quiere** :(y me lo robó por maldad.
- Los **sospechosos** son la **gente con la que comparto oficina**.

Mati (de la oficina 5) y
Vir (de la oficina 2)
mencionaron que tenían
muchísima hambre.

La Oficina 1 del DCIC
la compartimos Yamil,
Fede, Mario y yo.

Fer (el mayordomo del DCIC)
y Yamil no me quieren.

Tienen 10' para definir este programa Prolog

Hint: Pueden usar los términos `culpable(X)`, `sospechoso(X)`, `tiene_motivo(X)`
`con_hambre(X)`, `no_quiere_a(X,Y)`, `comparte_oficina_con(X,Y)`



Reglas y Hechos - Actividad

```
% Hechos
con_hambre(vir) .
con_hambre(mati) .
no_quiere_a(fer,diego) .
no_quiere_a(yamil,diego) .
comparte_oficina(yamil,diego) .
comparte_oficina(fede,diego) .
comparte_oficina(mario,diego) .

% Reglas
culpable(X) :-
    sospechoso(X) ,
    motivo(X) .

sospechoso(X) :-
    comparte_oficina(X,diego) .

motivo(X) :-
    con_hambre(X) .
motivo(X) :-
    no_quiere_a(X,diego) .
```

Tarea: Verificar que **yamil** es el culpable usando el intérprete [SWI-Prolog](#).

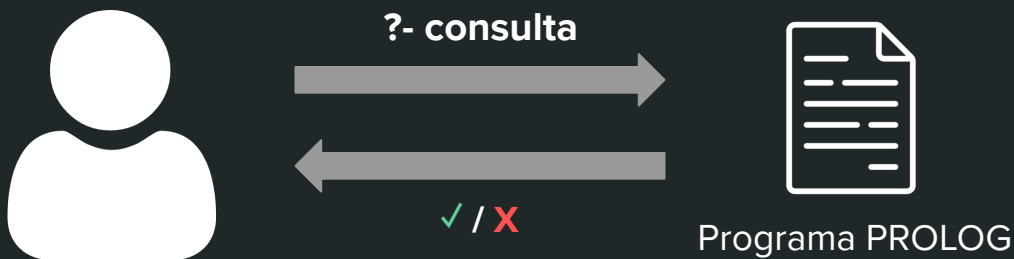


Programa PROLOG



Que hago con un programa PROLOG?

- Como vimos antes, un programa PROLOG me permite describir información o conocimiento del dominio que quiero modelar a través de reglas y hechos.
- Luego, **puedo realizar consultas a dicho programa** para verificar qué información “vale” teniendo en cuenta el conocimiento expresado.
 - Estas consultas se denotan como **?- T1, T2, ..., Tn.** donde cada **T** es un término.



A modo de ejemplo, verifiquemos quien fue el culpable en el caso anterior (**?- culpable(X)**)



Consultas PROLOG

Estamos diseñando una nueva red social *Unstagram*, en la cual los usuarios se pueden seguir entre sí, el conocimiento se almacena en el siguiente programa PROLOG:

```
sigue(mati,joaquin).  
sigue(fede,mati).  
sigue(mati,fede).  
sigue(fede,yamil).
```

“Fede sigue a Mati.”

```
amigos(X,Y):-  
    sigue(X,Y),  
    sigue(Y,X).
```

“Si X sigue a Y, e Y sigue a X, entonces son amigos.”

```
conocido(X,Y):-  
    sigue(X,Z),  
    sigue(Z,Y).
```

“Si X sigue a Z, y Z sigue a Y, entonces X conoce a Y.”



Consultas PROLOG

Hagamos un par de consultas

¿Joaquin sigue a Fede?

```
?- sigue(joaquin,fede) .  
false
```

¿Son Mati y Fede amigos?

```
?- amigos(mati,fede) .  
true
```

¿Existe alguien que siga a Mati? ¿Quien?

```
?- sigue(X,mati) .  
X = fede
```

¿Quien es conocido de Mati?

```
?- conocido(mati,X) .  
X = mati
```

```
sigue(mati,joaquin) .  
sigue(fede,mati) .  
sigue(mati,fede) .  
sigue(fede,yamil) .
```

```
amigos(X,Y) :-  
    sigue(X,Y) ,  
    sigue(Y,X) .
```

```
conocido(X,Y) :-  
    sigue(X,Z) ,  
    sigue(Z,Y) .
```



*¿No debería decir **X = yamil**?*
(esto lo veremos en detalle más adelante)

Parte II: Mirando PROLOG en profundidad

Variables y unificación

Como vimos anteriormente, los hechos y reglas pueden relacionar **variables y constantes**.

Consideremos la siguiente regla:

```
regla_magica(X) :-  
    X = 2,  
    X = 3.
```

Al ejecutar la consulta

```
?-regla_magica(X)
```

El intérprete Prolog nos devuelve **false**.

¿Por qué ocurre esto?

Variables y unificación

A diferencia de otros lenguajes de programación (Java, Pascal), la asignación de valores se hace a través de un **proceso de unificación**.

Intuitivamente, la unificación consiste en “**hacer coincidir**” dos términos (variables, constantes, estructuras) siguiendo estas reglas:

Una variable se puede unificar con cualquier cosa, siempre que la misma no tenga un valor asignado.



$$x = 2$$

Dos constantes se pueden unificar si son idénticas.



$$a = a$$

Dos estructuras se unifican solo si tienen el mismo nombre, el mismo número de argumentos, y cada par de argumentos se puede unificar.



$$f(x, a) = f(1, a)$$

Variables y unificación

La unificación de variables se efectúa de dos maneras:

1. Utilizando el operador de unificación =

```
?- X = 1           true
?- X = Y           true
?- X = a, Y = b, X = Y  false
```

2. En la unificación de predicados.

```
alumno(jose).
materia(jose,lfya).
...

?- alumno(X).
```

alumno(X)
↕
alumno(jose)

Unificación X = jose

Variables y unificación - Actividad

1. Una variable se puede unificar con cualquier cosa, siempre que la misma no tenga un valor asignado.
2. Dos constantes se pueden unificar si son idénticas.
3. Dos predicados se unifican solo si tienen el mismo nombre, el mismo número de argumentos, y cada par de argumentos se puede unificar.

$X = 5.$

true

$f(X) = f(\text{hola}, \text{chau})$

false

$\text{Var1} = \text{Var2}.$

true

$X = 3 + 2$

true

$X = \text{hola}, X = \text{chau}.$

false

$X = Y + 4, Y = 2$

true

$f(X) = f(6).$

true

$X = 1 + 1, X = 2$

false



Variables y unificación

De manera análoga, el operador $\backslash=$ nos permite verificar si 2 términos **no son unificables**.

Por ejemplo:

- a. $X \backslash= 4.$ **false** Porque X se puede unificar con 4
- b. $f(X,Y) \backslash= f(a,b,c).$ **true** Porque las estructuras f tienen distinta cantidad de parámetros.
- c. $f(a,X) \backslash= f(b,Y).$ **true** Porque si bien ambas f tienen misma cantidad de parámetros, a **no** se puede unificar con b.
- d. $X = f(A,B), A = 1, B = 2, f(2,1) \backslash= X$ **true** Porque luego de las primeras unificaciones, $X = f(1,2)$, que no se puede unificar con $f(2,1)$.

Operador IS

El operador IS nos permite unificar variables con **expresiones numéricas**:

“X is Y nos devuelve true si X se unifica con el valor resultante de evaluar la expresion Y”

Veamos algunos ejemplos:

```
?- X is 3+1.      true.  
?- 5 is 2+2.     false.  
?- X is Y+2      error!
```

```
?- X = 3+1, X is 4.  
?- X is 3+1, X = 4.  
?- X = 2,  
   Y = 1,  
   Z = (X+Y)+3,  
   W is Z.
```



En este último caso, ¿Qué valor tiene Z? ¿y W?

Agregando conocimiento, assert y retract

PROLOG nos permite **agregar y quitar conocimiento** durante la ejecución de una consulta, para ello usamos los siguientes predicados:

- **assert(X)**: Agrego X al programa PROLOG.
- **retract(X)**: Quito X del programa PROLOG.
(Donde X es un hecho Prolog.)

Ejemplo:

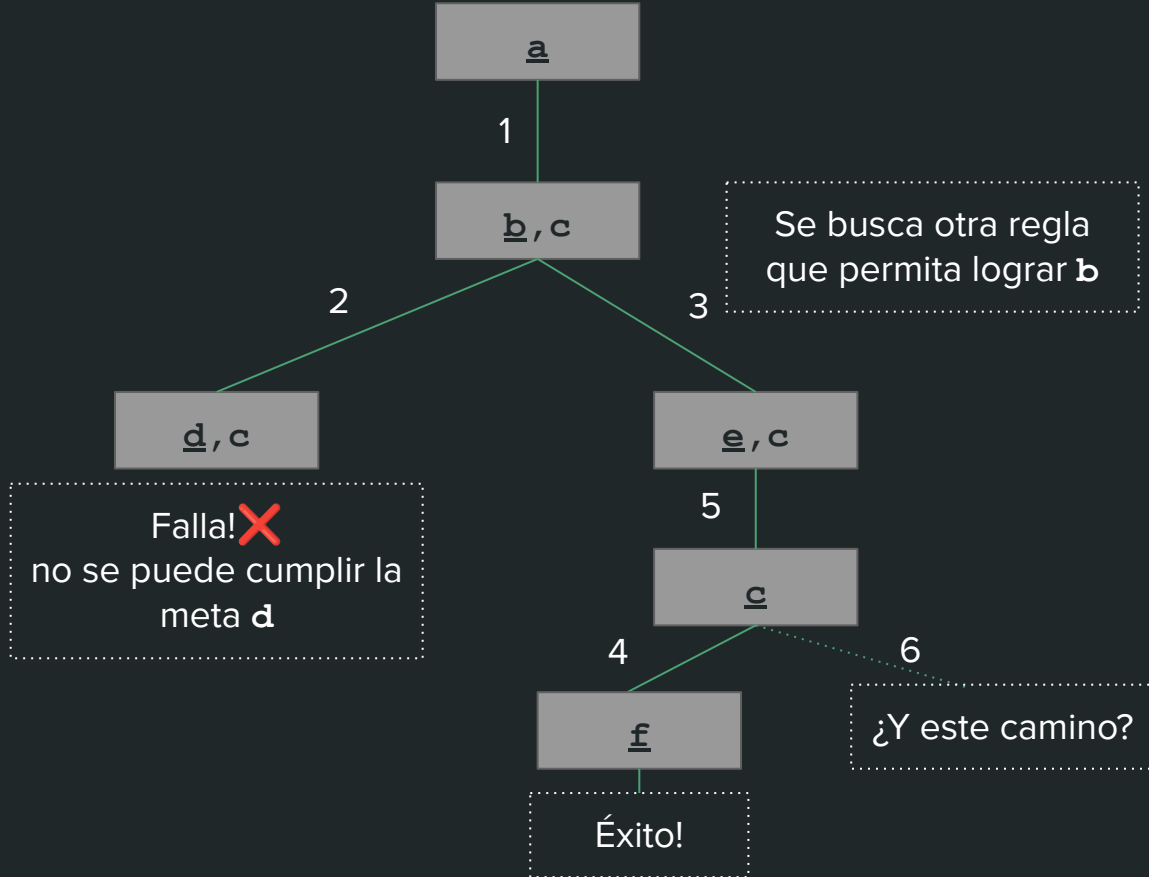
```
conectada(X,Y) :- viaje(X,Y) .  
conectada(X,Y) :-  
    viaje(X,Z) ,  
    conectada(Z,Y) ,  
    assert(viaje(X,Z)) .
```

¿Cómo funciona PROLOG? Resolución, Backtracking

```
1 a :- b, c.  
2 b :- d.  
3 b :- e.  
4 c :- f.  
5 e.  
6 c.  
7 f.
```

?- a.

La evaluación se hace de arriba hacia abajo y de izquierda a derecha.



Hasta aca llegamos por hoy!

Preguntas? (En Moodle quedan publicadas las diapos + Extras)

Negación en PROLOG

Tenemos un robot que circula por la ciudad, antes de cruzar una calle, quiere verificar que la misma no es peligrosa. Para ello, utilizando el siguiente programa Prolog:

```
puedo_cruzar:-  
    not(vienen_autos).
```

Hace la siguiente consulta:

```
?- puedo_cruzar.
```

La cual da como respuesta **true**..

Es decir, el robot pareciera estar a salvo...



Negación en PROLOG

Tenemos un robot que circula por la ciudad, antes de cruzar una calle, quiere verificar que la misma no es peligrosa. Para ello, utilizando el siguiente programa Prolog:

```
puedo_cruzar:-  
    not(vienen_autos).
```

Hace la siguiente consulta:

```
?- puedo_cruzar.
```

La cual da como respuesta **true**...

Es decir, el robot pareciera estar a salvo...



Sin embargo, cuando cruza la calle, **viene un auto, lo choca y es destruido**

¿Qué pasó?



Negación en PROLOG

PROLOG utiliza una forma de negación diferente a la negación clásica $\neg$, denominada **Negación por falla (not)**.

*Sea P un predicado, $\text{not}(P)$ tendrá éxito cuando P **no pueda probarse**
(**No significa que P sea falso! solo que no lo puede probar**)*

Veamos un ejemplo:

```
a :- b, c.  
b :- d.  
b :- e.  
c :- f.  
e.  
c.
```

```
?- a  
true  
?- not(a)  
false  
?- not(g)  
true
```

Muchas gracias por su atención
:)

Preguntas?